

Guia de Prática Laboratorial: API de Estoque

Metodologia: Trunk-Based Development, Git Submodules e Arquitetura MVC

Alvo: Preparação de ambiente Bare-Metal para provisionamento via Terraform e LocalStack

Estrutura da Atividade

Os alunos construirão o sistema em ciclos muito curtos, enviando o código diretamente para a branch `main` após cada etapa, simulando a integração contínua (Trunk-Based Development).

A arquitetura final conterá três repositórios:

- **estoque-contracts** (Submódulo de regras)
- **estoque-infra-config** (Submódulo de ambiente)
- **estoque-api** (API Core em Node.js)

FASE 1: Construção dos Submódulos

Os submódulos representam equipes externas e configurações de infraestrutura. Eles devem ser criados primeiro.

1. Repositório de Contratos (estoque-contracts)

Define as regras do negócio que o front-end e o back-end devem respeitar.

Passos:

1. Crie a pasta `estoque-contracts`, inicialize o Git (`git init`) e renomeie a branch para `main`.
2. Crie o arquivo `regras_produtos.json`:

```
{
  "unidades_medida": ["UN", "KG", "CX"],
  "alerta_estoque_baixo": 10
}
```

Commit (Trunk-Based):

```
git add regras_produtos.json
git commit -m "chore: cria contrato de regras de negocio"
git push -u origin main
```

2. Repositório de Infraestrutura (estoque-infra-config)

Define os parâmetros do LocalStack que serão usados pelo Terraform.

Passos:

1. Crie a pasta `estoque-infra-config`, inicialize o Git e garanta a branch `main`.
2. Crie o arquivo `ambiente-dev.json`:

```
{
  "sqs_url": "http://localhost:4566/000000000000/estoque-queue",
  "features": {
    "usar_sqs": false
  }
}
```

Commit (Trunk-Based):

```
git add ambiente-dev.json
git commit -m "chore: cria variaveis do ambiente localstack"
git push -u origin main
```

| FASE 2: Setup da API Core e Injeção dos Submódulos

Aqui começa o desenvolvimento da API principal. Tudo será commitado na main.

Passos Iniciais:

Crie a pasta `estoque-api`, inicialize o Git (main) e inicie o projeto Node:

```
npm init -y
npm install express
echo "node_modules/" > .gitignore
```

Adicione os submódulos:

```
git submodule add <URL_DO_REPO_CONTRACTS> contracts
git submodule add <URL_DO_REPO_INFRA> config
```

Crie o servidor base `server.js` :

```
const express = require('express');
const app = express();

app.use(express.json());

app.listen(3000, () => console.log('API rodando na porta 3000'));
```

Commit do Lote 1:

```
git add .
git commit -m "feat: setup inicial e importacao dos submodulos"
git push origin main
```

| FASE 3: Ciclo MVC 1 - Entidade Usuário

O objetivo desta fase é mostrar o fluxo de codificação em pedaços pequenos.

Etapa 3.1: Model de Usuário (Dados)

Crie a pasta `src/models` e o arquivo `userModel.js` :

```
// Banco de dados em memória para simulação
const users = [
  { id: 1, nome: "Admin", role: "gerente" },
  { id: 2, nome: "Operador", role: "estoquista" }
];

module.exports = {
  getAll: () => users,
  getById: (id) => users.find(u => u.id === id)
};
```

Commit do Lote 2:

```
git add src/models/userModel.js
git commit -m "feat: adiciona model de usuarios"
git push origin main
```

Etapa 3.2: Controller de Usuário (Lógica)

Crie a pasta `src/controllers` e o arquivo `UserController.js` :

```
const userModel = require('../models/userModel');

module.exports = {
  listarUsuarios: (req, res) => {
    const usuarios = userModel.getAll();
    res.json(usuarios);
  }
};
```

Commit do Lote 3:

```
git add src/controllers/userController.js
git commit -m "feat: adiciona controller para listagem de usuarios"
git push origin main
```

Etapa 3.3: View/Rotas de Usuário (Endpoints)

Crie a pasta `src/routes` e o arquivo `userRoutes.js`:

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

router.get('/', userController.listarUsuarios);
module.exports = router;
```

Atualize o `server.js` para usar a rota:

```
const express = require('express');
const userRoutes = require('./src/routes/userRoutes'); // NOVA LINHA
const app = express();

app.use(express.json());
app.use('/api/usuarios', userRoutes); // NOVA LINHA

app.listen(3000, () => console.log('API rodando na porta 3000'));
```

Commit do Lote 4:

```
git add src/routes/userRoutes.js server.js
git commit -m "feat: integra rotas de usuario na api"
git push origin main
```

FASE 4: Ciclo MVC 2 - Entidade Estoque (Uso dos Submódulos)

Aqui o sistema fará a leitura real dos submódulos e utilizará uma Feature Flag.

Etapa 4.1: Model de Estoque

Crie `src/models/estoqueModel.js`:

```
const fs = require('fs');
const path = require('path');

// Consumindo o Submódulo 1
const regrasPath = path.join(__dirname, '../../../contracts/regras_produtos.json');
const regras = JSON.parse(fs.readFileSync(regrasPath, 'utf8'));

let inventario = [
  { sku: "PROD-1", qtd: 15 }
];

module.exports = {
  obterEstoque: () => inventario,
  verificarAlerta: (qtdAtual) => qtdAtual <= regras.alerta_estoque_baixo
};
```

Commit do Lote 5:

```
git add src/models/estoqueModel.js
git commit -m "feat: cria model de estoque lendo regras do submodulo"
git push origin main
```

Etapa 4.2: Controller de Estoque (Com Feature Flag)

Crie `src/controllers/estoqueController.js`:

```

const estoqueModel = require('../models/estoqueModel');
const fs = require('fs');
const path = require('path');

// Consumindo o Submódulo 2
const configPath = path.join(__dirname, '../../config/ambiente-dev.json');

module.exports = {
  verificarStatus: (req, res) => {
    const config = JSON.parse(fs.readFileSync(configPath, 'utf8'));
    const estoque = estoqueModel.obterEstoque();

    let alertasEnviados = 0;

    // Uso da Feature Flag
    if (config.features.usar_sqs) {
      estoque.forEach(item => {
        if (estoqueModel.verificarAlerta(item.qtd)) {
          console.log(`Enviando para o LocalStack SQS: ${config.sqs_url}`);
          alertasEnviados++;
        }
      });
    }

    res.json({
      estoque,
      alertas_sqs_disparados: alertasEnviados,
      status_localstack: config.features.usar_sqs ? "ATIVO" : "INATIVO"
    });
  }
};

```

Commit do Lote 6:

```

git add src/controllers/estoqueController.js
git commit -m "feat: controller de estoque com feature flag do localstack"
git push origin main

```

Etapa 4.3: View/Rotas de Estoque

Crie `src/routes/estoqueRoutes.js`:

```
const express = require('express');
const router = express.Router();
const estoqueController = require('../controllers/estoqueController');

router.get('/status', estoqueController.verificarStatus);
module.exports = router;
```

Atualize o **server.js** :

```
// Adicione as linhas referentes ao estoque
const estoqueRoutes = require('./src/routes/estoqueRoutes');
app.use('/api/estoque', estoqueRoutes);
```

Commit do Lote 7:

```
git add src/routes/estoqueRoutes.js server.js
git commit -m "feat: integra endpoint de status do estoque"
git push origin main
```

| FASE 5: Automação (Ponte para o Terraform)

Para que a máquina criada pelo Terraform consiga subir essa API sem uso de Docker, é necessário um script de Bootstrap.

Crie o arquivo **install.sh** na raiz da API (estoque-api):

```
#!/bin/bash

# Atualiza pacotes e instala Node.js
sudo apt-get update -y
sudo apt-get install -y nodejs npm git

# Garante que os submódulos sejam baixados corretamente
git submodule update --init --recursive

# Instala as dependências da API
npm install

# Inicia a API em background (Porta 3000)
nohup node server.js > api.log 2>&1 &
```

Dê permissão de execução (simulação local):

```
chmod +x install.sh
```

Commit Final (Lote 8):

```
git add install.sh  
git commit -m "infra: script de automacao para provisionamento via terraform"  
git push origin main
```